BABEŞ-BOLYAI UNIVERSITY CLUJ-NAPOCA
FACULTY OF MATHEMATICS AND COMPUTER SCIENCE
SPECIALIZATION Artificial Intelligence

DIPLOMA THESIS

Real-Time Intrusion Detection for Web Applications Using Deep Learning

Supervisor
Alexandra–Ioana Albu

Author
Rares Marta

2026

ABSTRACT

This thesis designs, implements, and evaluates a real-time intrusion detection system (IDS) for IoT networks using deep learning on the CICIoT2023 benchmark dataset, delivering a complete, reproducible pipeline from raw packet captures to live per-flow classification.

The system is built around a two-hidden-layer Multi-Layer Perceptron (MLP) trained at binary and 8-class (attack-family) granularities on a temporally ordered train/validation/test split, so that evaluation reflects deployment conditions rather than optimistic random sampling. The MLP is compared against Random Forest and XGBoost baselines under identical conditions. On the binary task all three models exceed the 0.85 weighted-F1 target, with the tree ensembles reaching $\approx$0.92 and the MLP $\approx$0.90; on the harder 8-class task the trees lead at $\approx$0.77 and the MLP attains $\approx$0.73, none reaching 0.85, a shortfall traced to the limits of the public transport-layer features for separating application-layer attack families. Feature analysis identifies the per-flow packet count, the HTTPS/HTTP indicators, and header length as the most discriminative features, and prunes 14 of the 39 near-zero-variance CICIoT2023 features to leave a 25-feature model. Temperature scaling calibrates the MLP's softmax confidences.

The practical component is a web demonstration, NEURAL IDS, pairing a FastAPI backend with a React dashboard that accepts pre-extracted feature CSVs or raw packet captures, classifies flows in under 10 milliseconds, and displays confidence scores and per-class probability breakdowns. It correctly flags a synthetic SYN-flood capture as an attack (every flow flagged as Attack, mean calibrated confidence 99.6%) and classifies benign browsing as Benign (98% of flows), validating the core classification functionality required by the project specification.

Contents

Chapter 1

Introduction

1.1 Context and Motivation

Intrusion Detection Systems (IDS) are a core defensive layer in modern networks, monitoring traffic to detect malicious behaviour such as denial-of-service attacks, reconnaissance scans, brute-force attempts, and data exfiltration. The classical approach is signature-based detection, in which traffic is matched against a database of hand-written rules describing known attacks; the widely deployed Snort system is the canonical example [12]. Signature matching is fast and precise on threats it already knows, but it has two structural weaknesses: it cannot recognise an attack for which no rule has been written, and the rule set must be maintained by hand as new threats appear.

These weaknesses have become more pressing with the growth of the Internet of Things (IoT). Modern deployments place large numbers of constrained, infrequently patched devices on the same networks as critical services, and the volume and diversity of the resulting traffic strain a detection model that depends on enumerating every attack in advance. The same devices are attractive targets: compromised IoT endpoints are routinely conscripted into large botnets and used to launch volumetric denial-of-service campaigns, which is reflected in benchmark data where denial-of-service and distributed denial-of-service flows dominate the attack distribution [11].

Machine learning, and deep learning in particular, offers an alternative that learns the statistical structure of traffic directly from data rather than from hand-written rules [5, 9]. A model trained on labelled flows can, in principle, generalise to variants of known attacks and flag anomalous behaviour without an exact signature. This premise has produced a large literature reporting strong benchmark accuracy for neural intrusion detection, including work that applies deep models to real-time web intrusion detection rather than offline analysis alone [2, 16].

1.2 The Research-to-Deployment Gap

A model that scores well on a benchmark dataset is not automatically a model that works in practice [4]. This thesis pays attention to three practical problems that are easy to overlook when only the benchmark score matters.

The first problem is how the data is split for evaluation. If the split is random, flows recorded seconds apart in the same capture session can end up on both sides of the train/test boundary. The model is then tested on traffic it has, in effect, already seen, and the measured accuracy is higher than what it would achieve on genuinely new traffic. The second problem is time. Network traffic changes as applications, user habits, and attack tools evolve [3], and a deployed model always works on traffic that is newer than its training data. An evaluation that ignores this order says little about how the model will behave once deployed. The third problem is trust in the reported confidence. A detector does not only output a label; it shows a confidence score next to it, and neural networks tend to report confidences that are higher than their actual accuracy unless those scores are explicitly calibrated [6]. On top of these three, a real-time system has a latency budget: classifying a flow has to be fast enough to be useful, something an offline experiment never has to prove.

These concerns shape the design of this work. The headline evaluation uses a temporally ordered split, so the test data is strictly newer than the training data. The neural model is compared against strong tree-based baselines under identical conditions. The confidence scores are calibrated before being shown to the user, and inference latency is measured against an explicit acceptance criterion.

1.3 Approach and Scope

The work is built on the CICIoT2023 dataset, a large-scale IoT intrusion-detection benchmark of over 46 million labelled flow records spanning 33 attack types, captured in a 105-device smart-home testbed [11]. The primary classifier is a Multi-Layer Perceptron (MLP) trained at two granularities: a binary attack-versus-benign task and an eight-class task over coarse attack families. Because the public CICIoT2023 features are tabular flow statistics rather than raw packets, the deep model is not assumed to be the best tool by default; recent evidence shows that well-tuned simple neural networks are competitive with gradient-boosted trees on tabular data [7], and the thesis tests this empirically by comparing the MLP against Random Forest and XGBoost baselines on the same splits. Beyond offline evaluation, the trained model is deployed behind a live demonstration in which scripted attacks are launched against a controlled target and the resulting flows are classified in near real time.

The scope is bounded in two respects. The system is trained and evaluated only

on the 39-feature public release of CICIoT2023, which is derived from packet headers and flow metadata and carries no payload features; detection of application-layer attacks, whose signal lies in the payload, is therefore expected to be limited, and the results chapter reports this rather than working around it. Generalisation beyond the smart-home setting of the dataset is not claimed.

1.4 Objectives and Contributions

The thesis makes six contributions: (1) a reproducible end-to-end pipeline from the raw CICIoT2023 captures to a live per-flow classifier, with deterministic seeding and a fixed artefact contract; (2) a temporal evaluation of an MLP against Random Forest and XGBoost baselines at binary and eight-class granularities; (3) a feature analysis that prunes the 39 public features to a 25-feature set by dropping near-zero-variance flags and redundant continuous columns; (4) confidence calibration of the deep model by temperature scaling [6], so the confidence shown to a user reflects empirical accuracy; (5) a deployed demonstration system, NEURAL IDS, comprising a containerised inference backend and a web dashboard, validated against both benign and synthetic-attack traffic; and (6) an account of where the approach falls short, particularly the eight-class attack-family task, where the transport-layer-only public features cap performance below the 0.85 weighted-F1 bar that the binary task meets.

1.5 Thesis Structure

The remainder of this thesis is organised as follows. Chapter 2 presents the theoretical background, covering intrusion detection systems, deep learning for traffic classification, and the CICIoT2023 dataset. Chapter 3 presents the machine learning methodology: the data pipeline, the split strategies, the model architecture and training procedure, and the evaluation protocol. Chapter 4 reports the experimental results of that methodology and discusses the findings. Chapter 5 then states the requirements and specification of the application built around the trained model, and Chapter 6 describes its design, implementation, and testing, including the verification of the core functionality. Chapter 7 concludes the thesis and outlines directions for future work.

1.6 Declaration on the Use of Generative AI Tools

Generative AI tools were used during the preparation of this thesis: AI coding assistants supported the development of the training pipeline and the demonstration application, and AI language models helped draft and revise portions of the text. All such output was reviewed, tested, and edited by the author, and every experimental result was produced by running the pipeline and verified independently of any AI tool. The author takes full responsibility for the content of the thesis, the correctness of the results, and the final wording.

Chapter 2

Theoretical Background

This chapter presents the theoretical foundations underlying the work developed in this thesis. It begins with an overview of intrusion detection systems and their role in network security, then introduces deep learning as a tool for traffic classification, with a focus on multi-layer perceptrons. Finally, it describes the CICIoT2023 dataset, which serves as the training and evaluation basis for the models developed in subsequent chapters.

2.1 Intrusion Detection Systems

An Intrusion Detection System (IDS) is a security mechanism that monitors network traffic or host activity to identify malicious behavior. As networks have grown in scale and complexity, traditional perimeter defenses such as firewalls have proven insufficient on their own: attackers routinely bypass static rules and exploit new vulnerabilities, while modern networks exhibit high throughput, heterogeneity, and continual change [9]. IDS complement these defenses by analyzing traffic patterns and flagging suspicious activity.

IDS can be broadly categorized into two paradigms based on their detection methodology: signature-based and anomaly-based systems.

2.1.1 Signature-Based Detection

Signature-based IDS, exemplified by tools such as Snort and Suricata, operate by scanning network traffic for predefined patterns of known malicious activity [12]. These signatures can describe byte sequences, header anomalies, or protocol violations. Signature-based systems are highly effective at detecting known attacks with high precision, and they produce few false positives when the signature database is well-maintained.

However, their static nature makes them fundamentally limited against novel threats. Zero-day exploits, polymorphic attacks, and previously unseen attack variants will not match any existing signature and will therefore pass undetected. Signature databases also require continuous manual updates to remain effective, which introduces an inherent lag between the emergence of a new attack and the availability of its corresponding signature.

2.1.2 Anomaly-Based Detection

Anomaly-based IDS take a different approach: instead of matching against known attack patterns, they learn a model of normal network behavior and flag deviations from this baseline as suspicious. This paradigm has the significant advantage of being able to detect previously unseen attacks, since any traffic that deviates sufficiently from the learned normal profile will trigger an alert.

The trade-off is a higher false positive rate. Unusual but legitimate traffic (such as a sudden spike in usage during a product launch, or an uncommon protocol used by a newly deployed application) can be incorrectly flagged as malicious. The sensitivity of anomaly-based systems depends heavily on the quality of the normal behavior model and on the threshold chosen to separate normal from anomalous activity.

Figure 2.1 illustrates the typical architecture of a real-time deep learning-based IDS, showing the end-to-end pipeline from raw packet capture to alert generation.

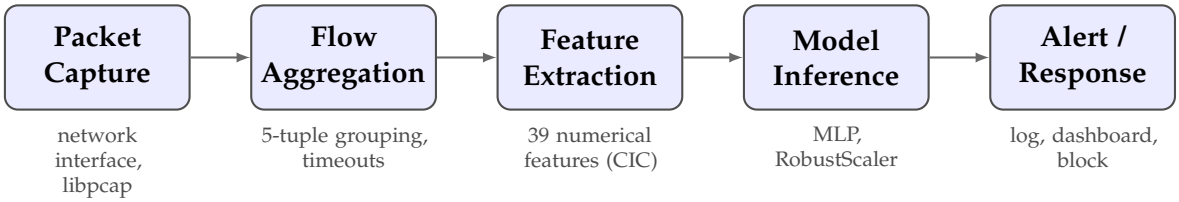

Figure 2.1: End-to-end architecture of a real-time deep learning-based intrusion detection system.

From a machine learning perspective, intrusion detection maps naturally to classification. In its simplest form, the task is binary: distinguish benign traffic from malicious flows. Multi-class setups are also common, with systems trained to recognize distinct categories of attacks such as DDoS, denial-of-service, reconnaissance, or botnet activity. The input is typically tabular flow data, where each flow is described by numerical features such as packet counts, byte rates, and TCP flag statistics [13].

2.1.3 Key Challenges

Deep learning models for IDS operate under several challenges that frequently undermine the transfer from benchmark performance to deployment. Class imbalance

is the most visible: benign traffic usually outnumbers attack traffic, and benchmark datasets reflect this. In CICIoT2023 DDoS attacks account for roughly 73% of all records while web-based attacks represent less than 0.1% [11], which can push a model to favour the majority class and makes accuracy a misleading metric unless weighted loss functions or stratified sampling are used. Concept drift compounds the problem: traffic distributions change over time as user behaviour, applications, protocols, and attacker strategies evolve, so a model trained on a static dataset may degrade once the traffic it sees diverges from its training distribution [3]. A third difficulty is dataset realism, since most benchmarks are generated in controlled testbeds rather than captured from production networks; testbeds allow precise labelling but may not capture encrypted communications, multi-stage attacks, or the noise of large-scale networks [9]. Finally, deployment constraints, latency and throughput budgets, integration with existing infrastructure, and long-term maintainability, are underreported in a literature focused on accuracy on static benchmarks [4].

2.2 Deep Learning for Network Intrusion Detection

Traditional machine learning approaches to IDS (such as Support Vector Machines, Decision Trees, and Random Forests) depend on manually engineered features and often struggle with high-dimensional, imbalanced, and non-stationary data. Deep learning offers an alternative by learning hierarchical representations directly from raw or lightly processed inputs. In the IDS domain, the literature reports many deep learning architectures, including deep feed-forward networks, convolutional neural networks (CNNs), recurrent neural networks (RNNs) with Long Short-Term Memory (LSTM) units, autoencoders, and more recently transformer-based models with self-attention [9].

This section focuses on the multi-layer perceptron (MLP), which serves as the primary model in this thesis, and briefly surveys other architectures for context.

2.2.1 Multi-Layer Perceptrons

A Multi-Layer Perceptron (MLP), also referred to as a fully connected feedforward neural network or deep neural network (DNN), is the foundational deep learning architecture for supervised classification. An MLP consists of an input layer, one or more hidden layers, and an output layer, where each layer is fully connected to the next.

Formally, given an input feature vector $\mathbf{x} \in \mathbb{R}^d$ representing a network flow

described by d numerical features, each hidden layer l computes:

$$\mathbf{h}_l = \sigma(\mathbf{W}_l \mathbf{h}_{l-1} + \mathbf{b}_l) \tag{2.1}$$

where $\mathbf{W}_l$ and $\mathbf{b}_l$ are the learnable weight matrix and bias vector of layer l, and σ denotes a non-linear activation function, typically the Rectified Linear Unit (ReLU), defined as $\text{ReLU}(x) = \max(0, x)$. The input to the first hidden layer is the feature vector itself: $\mathbf{h}_0 = \mathbf{x}$.

For binary classification (benign vs. attack), the output layer applies a sigmoid activation:

$$\hat{y} = \sigma_{\text{sig}}(\mathbf{w}^T \mathbf{h}_L + b) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{h}_L + b)}} \tag{2.2}$$

where $\mathbf{h}_L$ is the output of the last hidden layer and $\hat{y} \in (0, 1)$ represents the predicted probability that the input flow is an attack. The model is trained by minimizing the binary cross-entropy loss:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \tag{2.3}$$

using backpropagation and an optimizer such as Adam [14].

MLPs are well suited to flow-level intrusion detection. Network flow features form fixed-length numerical vectors, exactly the input format MLPs expect, and unlike CNNs (which assume spatial structure) or RNNs (which assume sequential structure) an MLP makes no structural assumption about the input, which fits independent flow records with heterogeneous features [7]. Inference is a sequence of matrix multiplications that modern hardware executes quickly, so a small MLP can process tens of thousands of flows per second on a single CPU core, which matters for low-latency detection [14]. Well-regularised MLPs are also competitive on tabular data: Kadra et al. showed that an MLP with a tuned combination of dropout, weight decay, and batch normalisation outperformed both XGBoost and specialised architectures such as TabNet on nearly half of the 40 datasets they evaluated [7]. In IDS research MLPs routinely reach 95–99% accuracy on standard benchmarks, which makes them a competitive baseline against which to measure more complex architectures [14, 5].

Training an MLP for IDS turns on a few standard choices. Class imbalance is handled by weighting the minority (attack) classes more heavily in the cross-entropy loss, or by oversampling methods such as SMOTE, or by focal loss that down-weights easy examples. Regularisation combines dropout (zeroing neuron outputs during training with probability typically between 0.2 and 0.5), weight decay, and early stopping on the validation loss. Feature scaling is essential because flow features

span very different ranges (packet counts versus flow durations in microseconds), so a scaler is fitted on the training set and applied unchanged to the validation and test sets.

2.2.2 Other Deep Learning Architectures for IDS

Several other architectures have been applied to intrusion detection. Convolutional networks reshape tabular features into one- or two-dimensional grids and use convolutions to extract local patterns tied to protocol-specific or timing signatures [15]. Recurrent networks with LSTM units capture temporal dependencies in sequential traffic, which helps in industrial or cyber-physical settings where context over time matters [8]. Autoencoders learn to reconstruct their input through a bottleneck and, when trained only on benign traffic, use the reconstruction error as an anomaly score; this needs no labelled attacks but depends heavily on the chosen threshold and on training-data quality [10]. Transformers use self-attention to model global relationships among features, treating each feature as a token, but they are usually supervised and more compute-intensive than the alternatives [16]. Compared with these, the supervised MLP used in this thesis needs labelled data and is less sensitive to genuinely novel attacks, but it trains and serves quickly and calibrates more easily than threshold-based anomaly detectors.

2.3 The CICIoT2023 Dataset

The CICIoT2023 dataset, published by the Canadian Institute for Cybersecurity (CIC) at the University of New Brunswick, is a large-scale benchmark dataset for IoT intrusion detection research [11]. It was selected for this thesis based on several criteria: recency (2023), scale (over 46 million records), diversity of attack types (33 distinct attacks across 7 categories), and growing adoption in the research community.

2.3.1 Testbed and Data Collection

The dataset was generated in a physical laboratory environment simulating a realistic smart home network. The testbed consists of **105 real IoT devices**, including smart cameras, speakers, plugs, lights, sensors, and home automation controllers from various manufacturers. Of these, 67 devices are directly involved in attack scenarios as either attackers or victims, while 38 Zigbee/Z-Wave devices are connected through 5 hubs.

The attack infrastructure uses 7 Raspberry Pi 4 devices configured as attackers. For distributed attacks (DDoS), multiple Raspberry Pis coordinate via an SSH-based

master-client configuration. Network traffic is passively captured using a Gigamon network tap and two monitoring stations running Wireshark, producing approximately 548 GB of raw PCAP data.

Feature extraction is performed using **DPKT**, a Python packet parsing library. Unlike CICFlowMeter (used in earlier CIC datasets such as CICIDS2017), DPKT operates at the packet level with a fixed-size packet window, computing features over a sequence of consecutive packets between two hosts rather than per-flow. This approach offers finer granularity and is more suitable for real-time detection, as features can be computed as soon as enough packets in the window are observed, without waiting for a flow to terminate [11].

2.3.2 Attack Taxonomy

The dataset contains **33 distinct attack types** organized into 7 categories, plus benign traffic, for a total of 34 class labels. Table 2.1 summarizes the attack categories, the number of attacks in each, and the tools used for generation.

Category	Count	Examples	Tools
DDoS	13	UDP Flood, SYN Flood, SlowLoris, HTTP Flood	hping3, golang-httpflood
DoS	4	TCP Flood, UDP Flood, SYN Flood, HTTP Flood	hping3, udp-flood
Reconnaissance	5	PingSweep, PortScan, OSScan, VulnerabilityScan	nmap, fping, vulscan
Web-Based	6	SQL Injection, XSS, Command Injection, Backdoor	DVWA, Remot3d, BeEF
Brute Force	1	DictionaryBruteForce	nmap, hydra
Spoofing	2	ARP Spoofing, DNS Spoofing	ettercap
Mirai	3	GREIP Flood, GREeth Flood, UDP-Plain	Adapted Mirai source

Table 2.1: CICIoT2023 attack taxonomy: 7 categories with 33 attack types.

2.3.3 Feature Set

The original CICIoT2023 paper describes a 46-feature representation produced by the DPKT-based extractor used during dataset construction. These features fall into eight

groups: three flow-timing features (flow duration, time-to-live, and inter-arrival time with the prior packet); three rate and throughput features (overall, source, and destination packet rates); two header and protocol features (header length and protocol type); seven binary TCP flag indicators (FIN, SYN, RST, PSH, ACK, ECE, CWR); five packet-count features (ACK, SYN, FIN, URG, RST counts); fourteen binary protocol indicators (HTTP, HTTPS, DNS, Telnet, SMTP, SSH, IRC, TCP, UDP, DHCP, ARP, ICMP, IPv, LLC); six packet-length statistics (sum, minimum, maximum, average, standard deviation, and individual length); and six aggregate statistics (packet count in flow, magnitude, radius, covariance, variance, and weight) derived from combined incoming and outgoing packet statistics.

At the time of writing, the publicly available CSV release distributed by the Canadian Institute for Cybersecurity contains a **39-feature subset** of the 46 features described in the paper. Specifically, the public CSV omits the source and destination rate features, the URG packet count, and four of the six aggregate statistics (*Magnitude*, *Radius*, *Covariance*, and *Weight*). The 39-feature version is the basis for all experiments and the live demonstration in this thesis. This is acknowledged as a deliberate scope decision: results in this work are not numerically directly comparable to papers that report on the full 46-feature variant, and that gap is part of the methodological story rather than an oversight.

2.3.4 Dataset Statistics and Class Distribution

The full dataset contains approximately 46.7 million records across 169 CSV files, about 13 GB uncompressed, and its class distribution is severely imbalanced. Benign traffic accounts for roughly 1.1 million samples (2.4% of the dataset); DDoS attacks make up about 73% of all records, with DDoS-ICMP_Flood alone contributing over 7.2 million samples; and the web-based attacks together amount to fewer than 25,000 samples, with the rarest classes (XSS, Backdoor_Malware, CommandInjection) holding only around 100 samples each.

The most extreme class ratio is approximately 5,751:1 between DDoS-ICMP_Flood and the smallest attack class. This imbalance must be addressed during training, whether by class weighting, stratified sampling, or both. Due to the dataset's size and computational constraints, most published experiments operate on stratified subsamples of 1–5 million records, preserving the original class distribution [11].

2.3.5 Comparison with Earlier Datasets

Table 2.2 compares CICIoT2023 with other commonly used IDS benchmark datasets.

CICIoT2023 offers several advantages over older benchmarks: it is significantly larger, contains more diverse and modern attack types (including IoT-specific attacks

Dataset	Year	Records	Features	Attack types	Domain
NSL-KDD	2009	150K	41 (custom)	4 classes	General
UNSW-NB15	2015	2.5M	49 (Argus+Bro)	9 classes	General
CICIDS2017	2017	2.8M	78 (CICFlowMeter)	14 attacks	Enterprise
CICIoT2023	2023	46.7M	39 (DPKT, CSV)	33 attacks	IoT smart home

Table 2.2: Comparison of commonly used IDS benchmark datasets.

such as Mirai botnet variants), and uses real IoT hardware rather than simulated traffic generators. Its primary limitations include the lack of encrypted traffic analysis, the restriction to a smart home IoT context (excluding industrial or medical IoT), and the fact that attacks are executed individually rather than as multi-stage campaigns [11].

2.3.6 Known Limitations

Several limitations of the CICIoT2023 dataset should be acknowledged. The attacks are executed in a controlled laboratory setting, so although real IoT devices are used, the dataset does not capture the way real attackers combine techniques, adapt, and conceal their actions. The features are extracted from packet headers and metadata: HTTPS is flagged as a binary indicator, but there are no features derived from encrypted-payload inspection, which limits generalisability as IoT traffic becomes increasingly encrypted. The testbed represents a single smart-home domain, so models may not transfer to industrial or healthcare IoT. The class imbalance is severe, with the most extreme ratio exceeding 5,000:1, and requires careful handling in both training and evaluation. Finally, the exact formulas for some derived features (Magnitude, Radius, Covariance, Variance, Weight) are not fully documented in the original paper.

Chapter 3

Machine Learning Methodology

This chapter presents the machine learning core of the thesis: how the raw CICIoT2023 captures are turned into clean training data, how that data is split so that the evaluation is trustworthy, how the model is built and trained, and how it is evaluated. Everything else in the system is built on top of the artefacts this chapter produces. The application that serves the trained model is specified in Chapter 5 and designed in Chapter 6; the experimental results of the methodology described here are reported in Chapter 4.

3.1 Data Pipeline

The training data pipeline performs a sequence of deterministic transformations from the raw per-folder CSVs to the train/validation/test tensors that feed the model. Each transformation is a separate pipeline phase so that intermediate state is inspectable.

3.1.1 Ingestion and Deduplication

For each of the attack folders, the pipeline lazily scans every CSV, projects only the feature columns plus a synthetic column that carries the source-capture filename, and concatenates the resulting lazy frames. Materialisation happens only after the projection, so the multi-tens-of-millions-of-rows corpus is never held in memory in its raw width.

CICIoT2023 contains a substantial fraction of exact duplicate rows, a known property of the dataset and a recurring concern in the literature. Deduplication is applied immediately after concatenation, before any sampling. Deduplicating before sampling is essential: a folder where most rows are duplicated would otherwise have the same row repeated many times in the sample, which would inflate accuracy on the corresponding test rows. The quantitative impact of each pipeline stage

(raw count, duplicates removed, unique rows, sampling cap applied) is reported in Chapter 4 (Figure 4.1).

3.1.2 Per-Class Subsampling

After deduplication, every folder whose deduplicated row count exceeds the per-class cap is randomly subsampled to that cap with a fixed seed. This brings the effective training-set size down from tens of millions of rows to a few million while keeping every attack class represented.

The cap is set at the high end of the range used in the CICIoT2023 literature. Random sampling is preferred over taking the leading rows of each folder because each attack folder is a chronological concatenation of several capture sessions: the first N rows come almost entirely from the first session, and a leading-rows sample would bias the model toward whatever happened in that opening session.

3.1.3 Caching

The deduplicated, subsampled, labelled rows are concatenated across folders and written to a single Zstandard-compressed columnar file. Subsequent runs skip ingestion and load from this cache directly. Regeneration is forced by deleting the cache file. This is documented inline so future runs do not silently use a stale cache after sampling logic changes.

3.1.4 Cleaning

The cleaning phase performs three operations on the materialised frame. First, null-label rows are dropped (defensive only; labels come from folder names, so this should be a no-op). Second, infinities are replaced with nulls; the actual median imputation is deferred to after the train/validation/test split so the medians are computed on the training partition only. Third, a $\log(1+x)$ transform is applied to the heavy-tailed continuous features (rates, total bytes, inter-arrival times). Binary protocol and flag indicators are excluded from the log transform.

3.1.5 Three Split Strategies

Three splitting functions are defined and selected by configuration, each addressing a different source of evaluation leakage.

Random row split. Two stratified splits in series produce 70/15/15 train/validation/test partitions, stratified by the fine-grained label so every class appears in every partition. This is reported only for parity with previously published numbers.

Per-capture split. Two consecutive group-aware splits with the source-capture filename as the group identifier produce a 70/15/15 split where no source capture ever appears in more than one partition. This eliminates within-session redundancy leaking across the boundary, but preserves no temporal order.

Temporal split (headline). For each attack folder, the source captures are sorted by filename and partitioned chronologically: the earliest 70% become train, the next 15% become validation, and the latest 15% become test. Folders with fewer than three distinct source captures fall back to a row-level temporal split inside the folder. This mirrors deployment (train on past, test on future) and is the headline result reported in the thesis. The temporal F1 is expected to sit a few points below the random F1; that gap is the honest generalisation story.

3.1.6 Train-Only Preprocessing Fit

For each split, column medians are computed on the training rows only, nulls in the train, validation, and test partitions are imputed using those medians, and the robust scaler is fitted on training rows alone before being applied to all three partitions. The robust scaler standardises each feature by its median and interquartile range rather than its mean and standard deviation:

$$x' = \frac{x - \mathrm{median}(x)}{\mathrm{IQR}(x)}, \qquad \mathrm{IQR}(x) = Q_3(x) - Q_1(x), \tag{3.1}$$

where Q_1 and Q_3 are the first and third quartiles estimated on the training rows. Centring on the median and dividing by the IQR makes the transform insensitive to the heavy upper tails of the CIC rate and byte-count features, which would otherwise dominate a mean/variance scaler. The contract that medians, quartiles, and the scaler are fitted strictly on training rows rules out test-set leakage and is what makes the temporal-split metrics meaningful.

3.2 Model Architecture and Training

3.2.1 MLP Topology

The model is a two-hidden-layer multi-layer perceptron with batch normalisation and dropout:

$$n_{\text{features}} \xrightarrow{\text{Linear}} 128 \xrightarrow{\text{BN, ReLU, Dropout}_{0.3}} 64 \xrightarrow{\text{BN, ReLU, Dropout}_{0.3}} n_{\text{classes}}$$

Batch normalisation, applied after each hidden linear layer, standardises that layer's pre-activations across the mini-batch to zero mean and unit variance before

rescaling them by two learned parameters (γ, β):

$$\mathrm{BN}(z) = \gamma \, \frac{z - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} + \beta, \tag{3.2}$$

where $\mu_{\mathcal{B}}$ and $\sigma_{\mathcal{B}}^2$ are the per-feature mean and variance over the current batch $\mathcal{B}$ and ϵ is a small constant for numerical stability.

The architecture is small for three reasons. Empirical studies of MLPs on tabular data report diminishing returns past two hidden layers when the input dimensionality is in the tens. Batch normalisation (Equation 3.2) is needed to stabilise training under the large class-weight range: at the fine-grained granularity the heaviest class weight is more than fifty times the lightest, and gradients without batch normalisation would oscillate. And the 0.3 dropout rate, more conservative than the common 0.5, generalises better on this dataset, where the cleaned training set is already several million rows.

3.2.2 Class-Weighted Cross-Entropy

Class weights are computed using the standard balanced-frequency rule applied to the encoded training labels, then passed to the cross-entropy loss. For a problem with K classes, N training samples, and n_c samples in class c, the weight of class c is

$$w_c = \frac{N}{K \, n_c}, \tag{3.3}$$

so that rare classes receive proportionally larger weight (and $\sum_c n_c w_c = N$). These weights enter the multi-class cross-entropy loss applied to the softmax outputs $\hat{y}_{i,k}$:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} \sum_{k=1}^{K} w_k \, y_{i,k} \log\big(\hat{y}_{i,k}\big), \tag{3.4}$$

where $y_{i,k}$ is the one-hot ground-truth indicator for sample i and class k. Equation (3.4) is the multi-class, class-weighted generalisation of the binary cross-entropy in Equation (2.3). Weighting is preferred over resampling because SMOTE would generate synthetic flow vectors that have no operational meaning (a half-and-half SYN-flood / DNS-spoofing flow does not correspond to any real attack), and weighted batch sampling would be redundant with weighted cross-entropy.

3.2.3 Optimiser, Schedule, and Early Stopping

Training uses Adam with an initial learning rate of 10^{-3}, paired with a learning-rate schedule that halves the rate after two consecutive epochs without validation-loss

improvement. A separate early-stopping mechanism with a patience of five epochs caps total training. The batch size is 4096, large enough to amortise the batch-normalisation overhead and small enough to keep the per-step gradient meaningful.

The training loop tracks training loss, validation loss, and the current learning rate per epoch, restoring the best validation-loss state at the end. The best state is what is serialised; the final-epoch state is discarded.

3.2.4 Hyperparameter Selection

The architecture and optimisation settings above are not arbitrary: they were selected by an automated hyperparameter search rather than fixed by hand. A Tree-structured Parzen Estimator (TPE) sampler explores the search space summarised in Table 3.1 over fifteen trials, each trained for ten epochs on a 100,000-row stratified subsample of the training set, with a median pruner terminating clearly unpromising trials early. The objective minimised is the validation loss. The search confirmed that a compact two-hidden-layer network with $[128, 64]$ units, 0.3 dropout, ReLU activations, the Adam optimiser, and a 4096 batch size is a strong configuration for this dataset; this configuration is then retrained to convergence (up to fifty epochs with early stopping) on the full training set and used for all reported results.

Hyperparameter	**Search range**
Hidden layers	2–4
Units per layer	64–512 (step 32)
Dropout	0.1–0.5
Activation	ReLU, ELU, LeakyReLU
Learning rate	10^{-4}–10^{-2} (log scale)
Optimiser	Adam, AdamW
Batch size	2048, 4096, 8192

Table 3.1: Optuna TPE search space for the MLP. Fifteen trials, ten epochs each on a 100,000-row subsample, validation loss as the objective, median pruning.

3.3 Evaluation Methodology

3.3.1 Classification Metrics

Every (split, granularity, model) combination is evaluated with the same protocol: accuracy, macro-averaged F1, support-weighted F1, macro precision and recall, the full per-class precision/recall/F1 report, and the row-normalised confusion matrix. Two averages of F1 are reported because they answer different questions under class imbalance. Weighted F1 averages the per-class F1 scores proportionally to

class support, so it reflects performance on the traffic mix as it actually occurs but can be dominated by the large DDoS and Mirai classes. Macro F1 weights every class equally, so it exposes weakness on the rare classes that weighted F1 can hide. Aggregate numbers are always accompanied by the per-class report, and the headline figures are taken from the temporal split.

3.3.2 Latency Benchmark

For each trained model, the benchmark loop runs $10{,}000$ inference passes per batch size in $\{1, 32, 256, 1024\}$ on CPU after a warmup of one hundred passes, with the timer wrapped around the feature scaling step *and* the model forward pass. The pipeline reports the p50, p95, and p99 of the per-batch latency in milliseconds and the implied flows-per-second throughput. The acceptance criterion, later formalised as NFR-1 (Section 5.4), is a sub-100 ms p95 end-to-end latency on a consumer laptop CPU at any of those batch sizes.

Chapter 4

Results and Discussion

This chapter reports the experimental results of the methodology presented in Chapter 3: data ingestion statistics, exploratory data analysis findings that shaped the feature set, model training dynamics, and classification performance across the two evaluated granularities (binary and 8-class attack-family) on the temporal split. It closes with a cross-model comparison and a discussion of the observed generalisation behaviour. The performance targets referenced here (NFR-2 and NFR-8) are formalised in the requirements of the application, in Section 5.4.

4.1 Data Ingestion and Class Distribution

The ingestion pipeline processed all 34 CICIoT2023 source folders and applied deduplication followed by per-class subsampling at a cap of 200,000 rows per fine-grained class. Figure 4.1 summarises the row counts at each stage.

Two features of this reduction matter. The duplicate fraction is unusually high: more than half of the raw records are byte-identical to another row in the same folder. This is a known property of the dataset and has been reported by other authors [11]. Deduplicating before sampling is therefore critical: without this step, each sampled row would carry implicit near-duplicate neighbours, inflating test accuracy to an optimistic and meaningless level. The sampling cap then bites unevenly: after deduplication the majority class (DDoS-ICMP_Flood) retains over 200,000 unique rows before the cap is applied, while the rarest web-based attack variants (XSS, Backdoor_Malware, CommandInjection) provide fewer than 5,000 unique rows each, well below the cap and therefore sampled in their entirety.

Figure 4.2 shows the class distribution after deduplication and sampling at the fine-grained 34-class level. Grouped into the eight attack families, the same working set is dominated by DDoS at 46.1%, followed by DoS (15.1%), Mirai (13.5%), and Reconnaissance (11.5%), with Benign at only 4.5% and the web-based family a

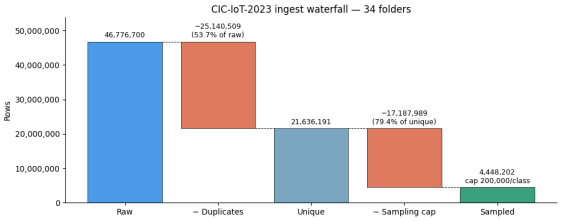

Figure 4.1: CICIoT2023 data reduction waterfall. Of 46,776,700 raw records, 53.7% are exact duplicates; the 200,000-row-per-class cap then discards 79.4% of the 21,636,191 unique rows, yielding a 4,448,202-row working set used for all splits and granularities.

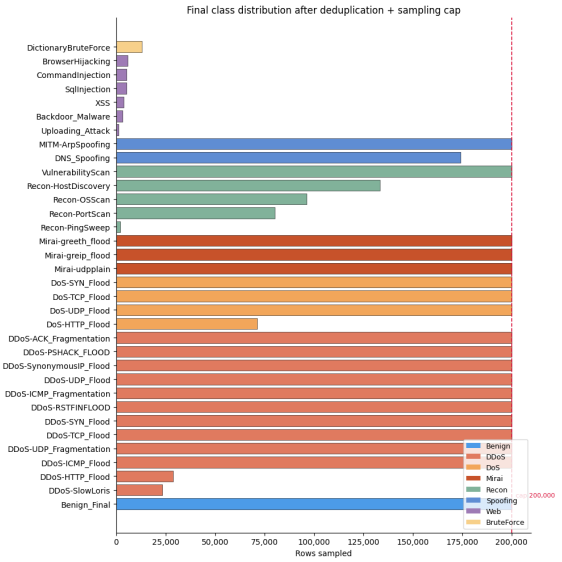

Figure 4.2: Final class distribution across the 34 fine-grained labels after deduplication and sampling; the dashed line marks the cap. The volumetric families sit at the cap, while the rare web-based attacks fall far below it, limited by the unique records available in the raw data.

thin sliver below 1%, which reflects the original dataset imbalance rather than the sampling cap.

4.2 Exploratory Data Analysis

Exploratory analysis on a 50,000-row stratified subsample of the training set informed two feature-set decisions: the removal of near-zero-variance binary features and the logging transform applied to heavy-tailed continuous features.

4.2.1 Continuous Feature Correlations

Figure 4.3 shows the Pearson correlation matrix of the 18 continuous features after the $\log(1 + x)$ transform.

Two perfectly collinear pairs are clearly visible: `AVG` packet length and `Tot_size` (total bytes, which is average $\times$ count), and `Std` (standard deviation of packet lengths) and `Variance` (its square). These pairs were retained because a RobustScaler absorbs collinearity without numerical instability, and removing one of each pair would not change model predictions for tree ensembles (which would select the same split point on either feature) and would make only a negligible difference for the MLP. The `IAT` (inter-arrival time) feature is nearly orthogonal to all others, which is consistent with its role as a timing feature rather than a size-based statistic.

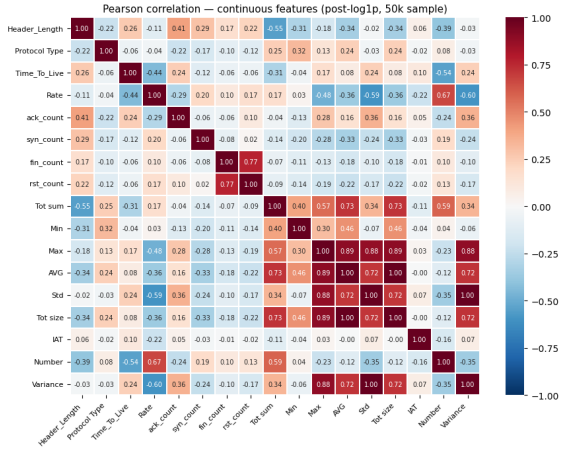

Figure 4.3: Pearson correlation matrix of the 18 continuous CICIoT2023 features on a 50,000-row post-log1p sample. Two pairs are perfectly correlated: `AVG` / `Tot_size` and `Std` / `Variance` ($r = 1.00$).

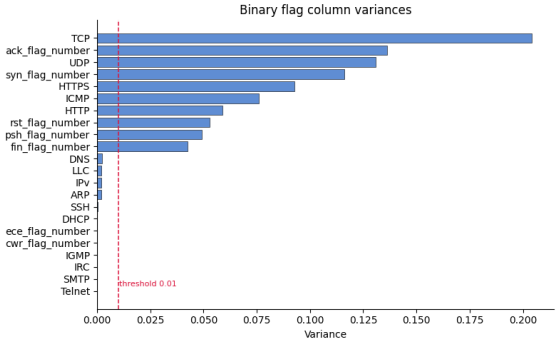

Figure 4.4: Empirical variance of the 21 binary flag and protocol-indicator features. The 14 features below the 0.01 threshold (dashed red line) are near-constant and are dropped, reducing the input dimensionality from 39 to 25.

4.2.2 Binary Flag Feature Variance

The 21 binary protocol-indicator and TCP flag columns vary widely in empirical variance. Figure 4.4 shows these variances in descending order.

Features with variance below 0.01 are close-to-constant across the training sample and contribute noise rather than signal: a near-constant feature effectively tells the model nothing about class membership. Fourteen such features are identified and removed. The remaining 25 features form the input vector used by all models, ensuring that the scaler and the MLP are not exposed to the collinear or near-zero-variance columns that would otherwise require special handling.

4.3 Feature Importance

Understanding *which* features drive a model's decisions is a recurring concern in the explainable-AI-for-IDS literature, where post-hoc attribution is used to make deep detectors auditable rather than opaque [1]. To understand which of the 25 retained features carry the most discriminative weight, permutation importance (the drop in model performance when a single feature's values are randomly shuffled, breaking its association with the label) was computed jointly on the Random Forest and XGBoost baselines and averaged. Figure 4.5 shows the top 20 features by this combined importance score.

The single most important feature by a wide margin is `Number`, the packet count within the flow window: scripted attack traffic produces highly regular packet counts that differ sharply from the variable counts of benign flows, making it the

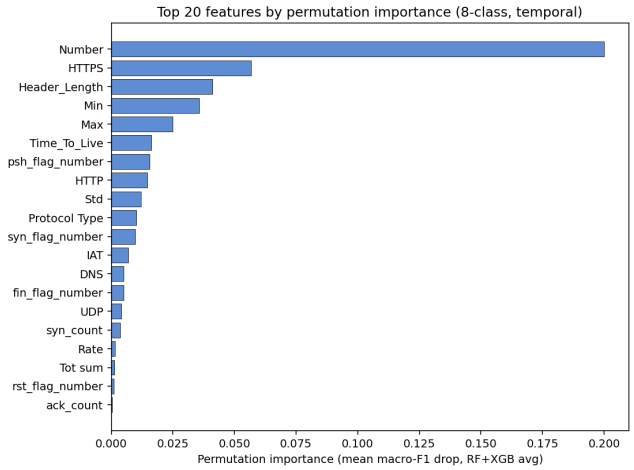

Figure 4.5: Top 20 features by average permutation importance (drop in macro-F1 under feature shuffling) across the Random Forest and XGBoost baselines (8-class, temporal split). The packet-count feature `Number` dominates by a wide margin, followed by the `HTTPS` protocol indicator, `Header_Length`, and the packet-length statistics `Min` and `Max`. The long low-importance tail confirms that the pruned 25-feature set carries little redundant signal.

strongest single discriminator across attack families. The `HTTPS` and `HTTP` protocol indicators rank next, separating web-oriented traffic from raw-socket floods, while `Header_Length` distinguishes protocol families at the coarsest level. The packet-length statistics (`Min`, `Max`, `Std`) and `Time_To_Live` contribute in the mid-range, capturing the size and network-hop signatures that help separate volumetric floods, and the TCP flag features (`psh_flag_number`, `syn_flag_number`) add handshake-pattern information. Because permutation importance here measures the drop in *macro*-F1, it rewards features that help the minority application-layer classes disproportionately, which is why the broadly-informative count and protocol-indicator features outrank individual flag counts.

4.4 Model Training Dynamics

Figure 4.6 shows the training and validation loss curves for the MLP on the temporal split at both the binary and 8-class granularities.

The loss curves show two effects worth noting. In the binary case, the validation loss oscillates in the early epochs before settling into a stable descent, reflecting the tension between the heavily weighted minority-class gradient signals and the majority-class (DDoS) signal. In the 8-class case, the loss inversion (validation below training) is a consequence of class-weighted loss: the training objective up-weights minority-class errors proportionally to their rarity, so the un-weighted validation loss (or equivalently, a validation set that happens to contain fewer minority-class hard examples) is lower by construction. The learning-rate schedule (halving on

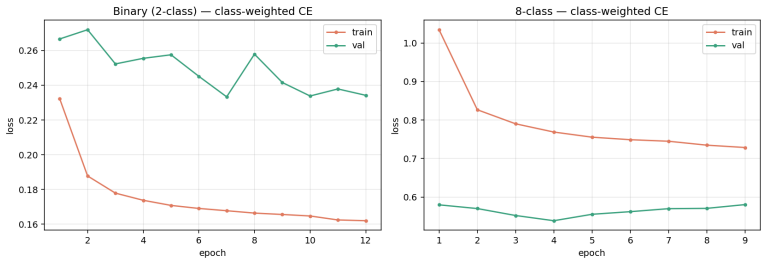

Figure 4.6: MLP training and validation loss (class-weighted cross-entropy) on the temporal split. Left: binary (2-class) granularity, reaching its best validation loss around epoch 7 and early-stopping at epoch 12. Right: 8-class granularity, best around epoch 4 and early-stopping at epoch 9. In both cases early stopping triggers at patience=5 after the best validation checkpoint is reached. Note that in the 8-class case the validation loss is consistently lower than the training loss: this inversion is expected under class-weighted cross-entropy when the minority-class weights impose a steeper penalty on training-set errors than on the validation set.

two consecutive non-improving epochs) is visible as the steeper descent segments followed by plateau phases.

4.5 Binary Classification Performance

Figure 4.7 shows the row-normalised confusion matrix for the MLP on the temporal-split binary (2-class) test set.

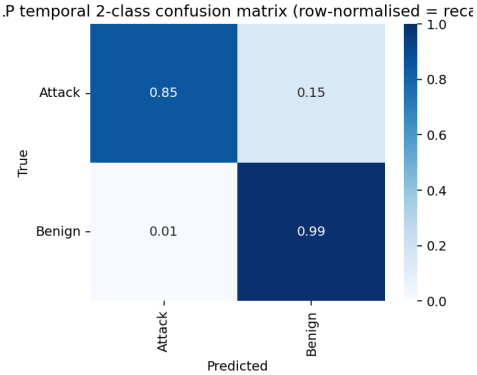

Figure 4.7: Row-normalised confusion matrix (per-class recall) for the MLP on the temporal-split 2-class test set: Attack recall = 0.85, Benign recall = 0.99.

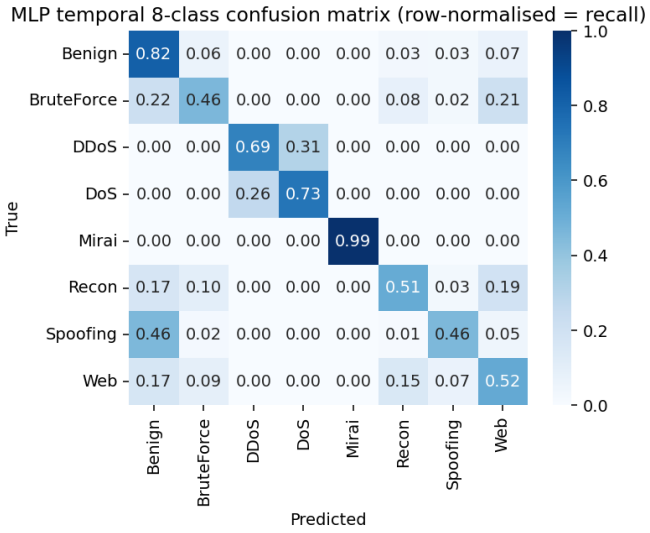

Figure 4.8: Row-normalised confusion matrix for the MLP on the temporal-split 8-class test set. Mirai is near-perfect (0.99); DDoS and DoS are frequently confused with each other; Spoofing and Recon bleed substantially into Benign.

The binary classifier achieves high Benign recall (0.99): fewer than 1% of legitimate flows are falsely flagged as attacks. Attack recall is lower at 0.85, meaning 15% of

malicious flows are missed. In a deployed IDS this trade-off is configurable via the classification threshold; the model's softmax output provides a continuous score for threshold tuning. The relatively modest attack recall under the temporal split (versus the higher values reported in random-split experiments in the literature) follows from evaluating on chronologically later captures the model has not seen, which is the setting a deployed detector actually faces.

4.6 Attack-Family Classification Performance

Figure 4.8 shows the row-normalised confusion matrix for the MLP on the temporal-split 8-class test set.

Table 4.1 summarises the per-class recall from Figure 4.8 alongside the main confusion targets.

Class	**Recall**	**Main misclassification**
Mirai	0.99	None (near-perfect)
Benign	0.82	small leakage to multiple classes (7% Web)
DoS	0.73	26% predicted as DDoS
DDoS	0.69	31% predicted as DoS
Web	0.52	17% Benign, 15% Recon
Recon	0.51	19% Web, 17% Benign
BruteForce	0.46	22% Benign, 21% Web
Spoofing	0.46	46% predicted as Benign

Table 4.1: Per-class recall on the temporal-split 8-class test set (MLP). Classes ordered by recall from highest to lowest.

The per-class recall splits into a strong and a weak group. Mirai stands apart, reaching 0.99 recall, the highest of any class, because the CICIoT2023 variants (Mirai-greith, Mirai-greip, Mirai-udpplain) all generate high-rate, fixed-payload floods over specific protocols and so produce an unusually homogeneous fingerprint that the model separates easily. The DDoS and DoS families, by contrast, are confused in both directions: both are volumetric floods differing mainly in the number of coordinated source hosts, and at the packet-feature level they share high rates, short inter-arrival times, and dominant SYN or UDP patterns, so the 31% DDoS-to-DoS and 26% DoS-to-DDoS confusion rates are unsurprising and match published results on similar datasets [9]. An operational IDS can tolerate this confusion, since both families call for the same response (rate-limiting and upstream filtering) and neither bleeds significantly into Benign.

The weaker classes are the application-layer and control-plane attacks. Spoofing (ARP and DNS spoofing, tied with BruteForce at 0.46 recall) has 46% of its flows predicted Benign, because ARP and DNS queries are operations that legitimate

devices also perform and the distinguishing signal is context and rate, which the fixed-size packet window captures imperfectly. Reconnaissance and Web attacks are partly confused with each other: port scans and host discovery on one side and HTTP-based injections on the other both rely on application-layer payloads invisible to the feature set, so probe requests that resemble malformed web requests produce the 19% Recon-to-Web and 15% Web-to-Recon leakage, and both also bleed into Benign. In short, the features describe how packets move, not what they carry, and the application-layer families are precisely the ones that distinction fails to capture.

4.7 Model Comparison

The same temporal split and evaluation protocol were applied to a Random Forest and an XGBoost baseline. Figure 4.9 compares weighted F1 across the three data partitions for all three models; accuracy and macro F1 follow the same train-to-test pattern and are reported in Table 4.2.

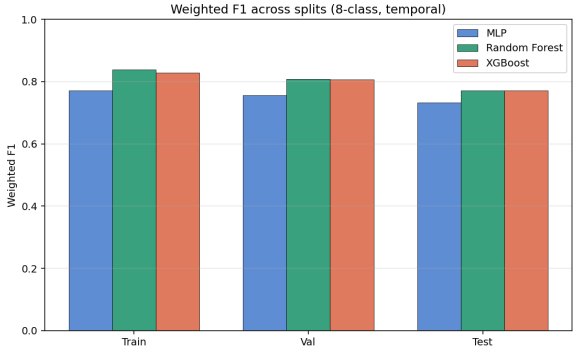

Figure 4.9: Weighted F1 across train, validation, and test partitions for the three models on the temporal split (8-class). All three degrade gently from train to test; the tree ensembles reach ≈0.771 against the MLP's ≈0.733 (full metrics in Table 4.2).

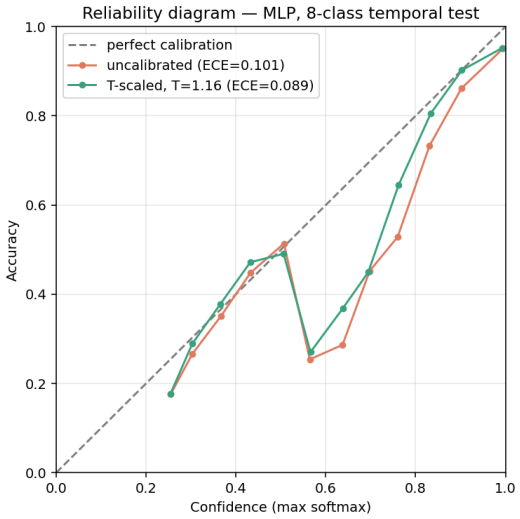

Figure 4.10: Reliability diagram for the MLP on the temporal-split 8-class test set, before and after temperature scaling. Points below the diagonal indicate overconfidence; scaling ($T = 1.16$) pulls the curve towards the diagonal, reducing ECE from 0.101 to 0.089.

Table 4.2 consolidates the key metrics.

A few observations follow from Table 4.2. First, no model reaches a weighted F1 of 0.85 at the 8-class granularity: the tree ensembles top out at ≈0.771 and the MLP at ≈0.733. The 0.85 acceptance bar (NFR-2, Section 5.4) is instead met comfortably on the binary task (Section 4.5), where weighted F1 reaches ≈0.90 for the MLP and ≈0.92 for the tree ensembles; the 8-class attack-family task is reported as the harder secondary

Model	Test Acc.	Test W-F1	Test Macro-F1	Val W-F1	Val Macro-F1
MLP	0.698	0.733	0.534	0.757	0.600
Random Forest	0.750	0.771	0.584	0.808	0.659
XGBoost	0.745	0.771	0.576	0.807	0.651

Table 4.2: Summary of key metrics on the temporal split (8-class granularity), held-out test set. W-F1 = weighted F1 (per-class F1 averaged by class support); Macro-F1 = unweighted (macro-averaged) F1, weighting every class equally. The large weighted-vs-macro gap reflects the dominance of the well-detected DDoS/Mirai classes in the support-weighted average.

objective. Its shortfall reflects what transport-layer features can and cannot express: they do not carry the application-layer signal these families differ by (Section 4.9). The model is not the limiting factor. Second, the tree-based models outperform the MLP by approximately 4 percentage points on weighted F1 and 5 on both accuracy and macro F1. This result is consistent with the growing body of evidence that gradient-boosted trees and random forests are competitive (and often superior) to MLPs on tabular tasks when the sample count is in the low millions [7]. The MLP remains the primary model for the demonstration system because its inference is implemented in a single portable PyTorch call, without the additional sklearn dependency; the tree baselines serve their designed role of confirming that the deep model is not strictly worse. Third, the macro F1 gap (MLP: 0.534 vs tree: 0.58) is comparable to the weighted F1 gap, and the low macro F1 of all three models confirms that the difficulty is concentrated in the minority classes (particularly Spoofing, Recon, BruteForce, and Web), while performance on the dominant DDoS and Mirai families (which drive weighted F1) is far stronger.

4.8 Confidence Calibration

The demonstration system reports the maximum softmax probability as a confidence score alongside each verdict. Raw softmax outputs, however, are known to be poorly calibrated (typically overconfident) on modern networks [6]. To quantify and correct this, the MLP is calibrated by *temperature scaling*: a single scalar $T > 0$ is fitted on the validation-set logits $\mathbf{z}$ by minimising the negative log-likelihood, and predictions are recomputed as $\mathrm{softmax}(\mathbf{z}/T)$. Temperature scaling leaves the argmax (and hence accuracy) unchanged; it only rescales confidence. Calibration quality is measured by the Expected Calibration Error (ECE), the support-weighted average gap between confidence and accuracy across fifteen equal-width confidence bins.

On the 8-class task the fitted temperature is $T = 1.16$, reducing ECE from 0.101 to 0.089, a modest correction indicating the class-weighted multiclass model was already close to calibrated. On the binary task the effect is larger: $T = 2.73$ lowers ECE from 0.108 to 0.080, reflecting the pronounced overconfidence that class-weighted training induces on the dominant attack class. Applying the fitted temperature in the serving path therefore yields confidence scores that more faithfully reflect empirical accuracy, at zero cost to the predicted labels. Residual miscalibration remains and per-class (vector) scaling is left as future work.

4.9 Discussion

The temporal split provides a meaningfully harder test than a random row split would, because the test captures were recorded later in each attack session and may reflect slightly evolved traffic patterns. Despite this, all three models generalise well, and the small train→test accuracy drops (below 8 percentage points) suggest that the CICIoT2023 attack sessions are reasonably stationary within each folder, a consequence of the controlled laboratory setting.

The principal failure modes are the ones the design anticipated, and Section 4.6 traced each to the feature set rather than the model: the application-layer attacks (Web, Recon, Spoofing) leave no signal in a transport-layer representation, and the statistically similar DDoS and DoS floods overlap in feature space. Both are properties of the dataset's extraction methodology, not model defects.

NFR-2 (at least 0.85 weighted F1 on the temporal *binary* test set) is satisfied by all three models, which reach ≈0.90 (MLP) to ≈0.92 (tree ensembles). At the 8-class granularity no model reaches 0.85 (the best attain ≈0.77), which the design treats as the honest ceiling imposed by a transport-layer feature set rather than a target failure, in keeping with the secondary status assigned to the 8-class task in NFR-2. NFR-8 (honest reporting) is respected by making the temporal split the headline result and by reporting per-class metrics rather than only aggregate numbers, so that the minority-class weakness is visible without requiring the reader to look past a high aggregate figure.

Chapter 5

Application Requirements and Specification

This chapter defines the application that constitutes the practical contribution of the thesis: a web demonstration in which the user submits captured network traffic and receives per-flow intrusion-detection verdicts from the model of Chapter 3. It identifies the actor and use cases, states the functional and non-functional requirements, and fixes the system-level specification that constrains the implementation in Chapter 6.

5.1 Project Goal and Scope

The deliverable is a real-time intrusion detection system (RT-IDS) for IoT networks built on CICIoT2023, structured as two cooperating subsystems. The training subsystem is the reproducible offline pipeline of Chapter 3, which serialises the small set of artefacts needed for inference. The serving subsystem is a web application that loads those artefacts, accepts either a pre-extracted feature CSV in CIC format or a raw packet capture (from which a flow-extraction tool derives the features), runs the trained model, and presents per-flow verdicts and an aggregated summary in the browser.

The system targets an interactive, single-request setting: the user supplies a sample of traffic and reviews the verdict on screen. Line-rate gateway protection (sub-millisecond per-packet inference at multi-gigabit throughput) is out of scope; this narrower sense of "real time" sets the latency thresholds below. Both classification granularities are exposed: the 2-class gate an inline deployment would use, and the 8-class attack-family view an analyst would consume.

5.2 Stakeholders and Actors

The application has a single actor, the *operator*: the person who opens the web interface, selects a model variant, uploads a packet capture or feature CSV, and reads the per-flow verdicts. During the thesis defence the author acts as operator, and a reviewer reproducing a result plays the same role. Model training is deliberately *not* a function of the application: users cannot train, retrain, or modify models through it. Models are trained offline by the author through the pipeline of Chapter 3, and the application only consumes the serialised artefacts that training produces. Regenerating those artefacts is a development-time activity that happens outside the application and exposes no user-facing functionality.

The core classification path is stateless: submitted captures and CSVs are processed in a temporary location and never persisted (FR-D5). The deployed frontend adds a thin convenience tier on top (user accounts and a per-user history), which stores only derived result metadata, never the uploaded traffic itself (Section 6.4.4).

5.3 Functional Requirements

The functional requirements are split into pipeline-side requirements (the offline training subsystem) and demo-side requirements (the web application).

5.3.1 Pipeline Requirements

The pipeline-side requirements formalise the methodology of Chapter 3, so they are stated here without repeating the rationale given there. Ingestion must be reproducible (FR-P1): re-running with the same seed and source files yields a byte-identical columnar intermediate, with exact duplicate rows dropped before any sampling. Per-class subsampling must be uniform under a configurable cap with a fixed seed (FR-P2), drawing randomly rather than taking leading rows. The pipeline must produce the three splits of Section 3.1.5 over the same subsampled rows (FR-P3) and, for each split, train and evaluate one model per granularity while reusing the same row indices (FR-P4). Imbalance is handled by class-weighted cross-entropy, with synthetic oversampling and weighted batch sampling explicitly rejected (FR-P5; Section 3.2.2). A Random Forest and a gradient-boosted-tree baseline must be trained on the same splits and reported under the same metrics (FR-P6). For every (split, granularity) pair the pipeline reports the metric set of Section 3.3.1 (FR-P7) and the percentile latency benchmark of Section 3.3.2 (FR-P8; a mean latency alone does not satisfy this requirement), and persists the artefact triple, the fitted scaler, the fitted label encoder, and the trained weights, under a naming convention uniform enough

that new variants need no code changes (FR-P9, Section 5.6.1).

5.3.2 Demo Requirements

The core demo requirement (FR-D1) is per-flow classification of submitted traffic: the demo accepts either a CSV in the CIC feature format or a raw packet capture, and returns a per-flow verdict table with an aggregated summary. For packet captures the established CIC flow-extraction tool is invoked as a subprocess; flow extraction is not reimplemented, since that would risk numerical drift between live and training features. The remaining requirements support this core. Model-variant selection (FR-D2) lists every (split, granularity) pair with trained artefacts on disk and lets the user switch without restarting. The output schema is deterministic (FR-D3): a summary line with the flow count and a per-label breakdown, followed by a table of row index, predicted label, and confidence. Input is self-validating (FR-D4): a CSV missing expected columns produces an error naming them, and a flow-extraction failure surfaces the tool's own error message rather than crashing the request.

Raw traffic is never persisted (FR-D5): uploads are processed in a temporary location, removed when the request ends, and never logged; only derived result metadata (model, predicted label, confidence, mode, timestamp) may be stored for an authenticated user. The application is deployed as a permanent public service (FR-D6) at a fixed HTTPS URL with no presenter-side configuration: the backend as a Docker container on Hugging Face Spaces, the React frontend on Vercel, and a local Gradio UI as fallback. The deployed frontend requires authentication and keeps a per-user history of past classifications (FR-D7, metadata only per FR-D5), with isolation enforced at the storage layer rather than in client code.

5.4 Non-Functional Requirements

End-to-end per-flow latency through the demo (CSV parsing, scaling, forward pass, result formatting) must stay below 100 ms at p95 on a consumer laptop CPU for batches of up to 1024 flows (NFR-1); packet-capture parsing is excluded because the flow-extraction tool dominates that path, and capture-to-verdict latency is reported separately. For classification performance (NFR-2), the temporal-split model must reach at least 0.85 weighted-F1 on the held-out binary test set; the harder 8-class task is a secondary objective whose weighted-F1 is expected to be lower, since the transport-layer features cannot fully separate the application-layer families.

The pipeline must be reproducible (NFR-3): a single deterministic seed feeds every random source, so re-running yields identical splits and metrics within ± 0.001 F1. It must fit within 16 GB of RAM at every stage (NFR-4), which forces lazy

columnar processing, and must run on CPU-only hardware (NFR-5), using a GPU opportunistically but never as a dependency. The demo must run on Windows and Linux (NFR-6) and the backend must be packageable as a Docker image. Operation must stay simple (NFR-7): a permanent HTTPS URL, a single command for local development, and no database, message queue, or dedicated model server, since the backend loads artefacts directly into the application process. Reporting must be faithful (NFR-8): headline metrics come from the temporal split even though the random split reports higher numbers, and per-class metrics always accompany aggregates so majority-class dominance cannot mask weak minority classes.

5.5 Use Cases

The principal interactions are summarised in Table 5.1; the core classification path is then narrated as the representative flow.

ID	Actor	Trigger	Outcome
UC-1	Operator	Uploads a packet capture	Per-flow verdict table and summary; temporary capture removed afterwards.
UC-2	Operator	Uploads a pre-extracted CIC CSV	Per-flow verdict table; missing columns reported as a readable error.
UC-3	Operator	Selects a different model card	Result screen re-rendered under the new variant with no restart or reload.
UC-4	Operator	Opens the public NEURAL IDS URL	Demo reachable from any network at a permanent URL with no presenter-side setup.

Table 5.1: Principal use cases with their actors, triggers, and outcomes.

In the core flow (UC-1), the operator opens the public NEURAL IDS URL, selects a model variant, and uploads a packet capture. The application stores the capture temporarily, runs the flow-extraction tool to obtain a feature CSV, applies the trained scaler and model, and renders the summary and the per-flow verdict table; the temporary files are then removed. If flow extraction fails, its error output is returned with an empty table. The CSV path (UC-2) is identical except that it validates the CIC column set instead of running flow extraction.

5.6 System Specification

This section fixes the high-level architecture and the technology choices; the detailed design follows in Chapter 6.

5.6.1 High-Level Architecture

The system consists of two pipelines that meet at a small set of serialised artefacts. The training pipeline runs offline and produces those artefacts. The serving pipeline is the live demo, which loads them and answers classification requests. Figure 5.1 shows the data flow.

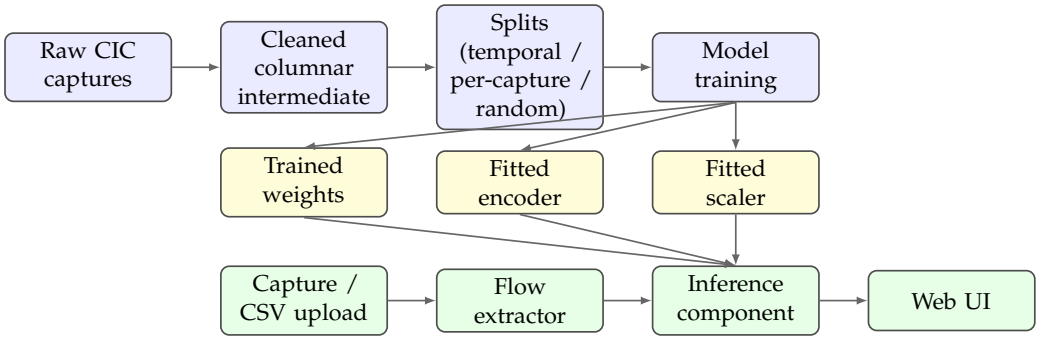

Figure 5.1: High-level architecture of the RT-IDS application. The training pipeline (blue) produces three serialised artefacts (yellow), which the serving pipeline (green) loads to answer per-flow classification requests.

5.6.2 Technology Stack and Rationale

The technology choices follow from the requirements. Data wrangling uses a lazy columnar engine with compressed intermediates, so the corpus is processed without being materialised in memory (NFR-4). Features are scaled with the robust median/IQR scaler and labels use a reversible integer encoder; imbalance is handled by class-weighted cross-entropy (Section 3.2.2). The deep model is the MLP of Section 3.2, compared against Random Forest and gradient-boosted-tree baselines. Packet captures are converted to feature CSVs by CICFlowMeter, the CIC team's reference tool, to avoid numerical drift between live and training features. The demo pairs a Gradio + FastAPI backend with the NEURAL IDS React dashboard, containerised on Hugging Face Spaces and served from Vercel respectively, which yields a permanent HTTPS URL. A single deterministic seed propagated to every random source provides reproducibility (NFR-3).

5.6.3 Constraints and Assumptions

Several constraints bound the work. The 39-feature public CSV release of CICIoT2023 is the only source of training data; the 46-feature variant used by some published comparisons was not available from the official distribution, so numerical results are not directly comparable across the two feature sets (Section 2.3.3). The dataset comes from a smart-home testbed, so generalisation to industrial, medical, or vehicular networks is not claimed, and its header-and-metadata features mean detection of application-layer web attacks is expected to be weak, a property of the feature set discussed in the results chapter. The serving target is a single-container backend with a static frontend, so distributed serving, load balancing, and high availability are out of scope. Authentication uses Supabase email/password, limited to identity verification with per-user isolation enforced by Postgres row-level security; the local-development Gradio UI does not authenticate. Finally, the deployed model is the one trained offline; the demo does not adapt to feedback during a session.

Chapter 6

Application Design, Implementation and Testing

This chapter covers the design and implementation of the application specified in Chapter 5 and the testing that verifies the core functionality (per-flow classification of submitted traffic). The machine learning core that the application serves (the data pipeline, the split strategies, and the model architecture and training procedure) was presented in Chapter 3; this chapter is about everything built around it. It is organised top-down, starting from the architectural breakdown, moving through each component's design and the implementation choices that turn it into running code, and finishing with the testing strategy and the functional acceptance evidence.

6.1 Architectural Overview

The system is split into a training subsystem (offline) and a serving subsystem (online), as introduced in Section 5.6. The two are decoupled by a small, well-defined contract: a fixed number of serialised artefacts per trained model variant. The decoupling serves two purposes. It lets the training side iterate freely (different splits, granularities, hyperparameters) without forcing changes to the serving code, and it lets the serving code stay tiny and observable.

6.1.1 Component Decomposition

The application has five components, each owning a single responsibility so that a fault is easy to localise. The boundary between them is the artefact contract rather than a network protocol or in-process API, which lets the training side be re-run independently of any deployed serving instance and the serving side be reloaded by pointing it at a different artefact directory. The training pipeline handles ingestion, deduplication, per-class subsampling, splitting, scaler and encoder fitting, model

training, evaluation, latency benchmarking, and artefact persistence. The inference component loads the scaler, encoder, and weights for a given (split, granularity) variant, preprocesses incoming feature rows, runs the model, and returns labelled predictions with confidences. The capture-to-CSV bridge locates a CICFlowMeter backend, runs it as a subprocess against a packet capture, and returns the path of the feature CSV it produces. The web application discovers the model variants on disk, exposes the user interface, routes uploads through the inference component, and formats the verdicts. Beneath all of these, the project specification fixes the invariants (feature set, split strategies, classification granularities, and modelling constraints) that bind every component.

6.1.2 Artefact Contract

The serving subsystem consumes exactly three artefacts per model variant, produced by the training pipeline and read back by the inference component: the fitted feature scaler for the split (one per split, shared across granularities because the input feature space is identical regardless of label encoding), the fitted label encoder for the (split, granularity) pair (one per granularity, because the label set changes), and the trained model weights for that pair.

The artefact filenames encode both the split and the granularity in a uniform pattern. As a consequence, the web application can probe the artefact directory at startup, parse each filename, and instantiate one inference component per complete artefact triple it finds, with no hard-coded list of variants. Exposing a newly trained variant in the user interface is therefore purely a question of dropping its three files into the artefact directory and restarting the application.

6.2 Inference Component Design

The inference component is the single seam between trained artefacts and live requests. Its constructor takes the artefact directory, the split name, and the granularity, and loads the matching triple. Its prediction method takes a frame of feature rows and returns the predicted labels, the per-prediction confidences, the full softmax probabilities, and the class-name list in the encoder's index order.

6.2.1 Preprocessing Mirror

The inference component must reproduce the training-time preprocessing exactly. Its preprocessing path therefore mirrors the training pipeline at three points. The expected feature columns are validated and reordered so that column ordering does

not silently shift between the trained scaler and the live data. Infinities are replaced with nulls and the $\log(1 + x)$ transform is applied to the continuous features only, leaving the flag and protocol indicators binary. Residual nulls are then filled with zeros: at training time median imputation handled nulls, but the trained scaler operates in the transformed space, so a residual null in a live request is filled with a constant rather than a re-derived median. The transformed array is passed through the scaler's transform method, never a re-fit, since the scaler is immutable at inference time.

6.2.2 Confidence and Label Decoding

The model emits logits; a softmax along the class dimension produces per-class probabilities; the argmax selects the predicted class index; the encoder's inverse transform maps the index back to the human-readable label. The maximum softmax probability is reported as confidence. This confidence is not a calibrated probability; it is shown because users expect a numeric score beside each verdict, and the maximum softmax value is a simple proxy for one.

6.3 Capture-to-CSV Bridge Design

The bridge component exists because the training data was produced by an established flow extractor. Reimplementing extraction in Python would risk numerical drift between the live and training distributions, so the project invokes the original tool instead. The component wraps two backends and returns the path of the feature CSV. The Java CICFlowMeter is preferred, detected through an environment variable pointing at a built installation and invoked through the launcher script for the host platform (one variant for Linux and macOS, one for Windows). If it is absent, the bridge falls back to a community Python port located through the executable search path, and if neither is available it raises an error naming both options. The two backends differ in output filename convention, so the bridge renames the produced file to a single canonical form and downstream code never branches on backend. Both are launched with their standard output and error captured, and a non-zero exit is translated into a domain-level error carrying that standard error, which is what surfaces under FR-D4 as the tool's own diagnostic message rather than a Python traceback.

6.4 Web Application Design

The serving subsystem is implemented as two cooperating layers: a **backend** (Gradio UI combined with a FastAPI REST endpoint, running in a single Python process) and a **React dashboard frontend** that calls the REST endpoint and provides a richer user experience. The two layers are decoupled through the REST API contract, which allows the frontend to be hosted independently on a public platform while the backend runs locally or on Hugging Face Spaces.

6.4.1 Backend: Gradio + FastAPI

At startup, the backend performs the filesystem-driven discovery described in Section 6.1.2: it globs the artefact directory for trained model weights, attempts to load each complete artefact triple, and builds a dictionary of ready predictors keyed by a human-readable `split / granularity` string. If no triples are found, the process exits with a diagnostic message.

The backend exposes two interfaces over the same port. The Gradio interface provides a lightweight local UI with a model-variant dropdown and two upload tabs (one for pre-extracted CSVs, one for raw packet captures). The FastAPI REST endpoint at `/api/classify` accepts a multipart form submission with the upload file, a model type label, the split name, the classification mode, and the input type (CSV or PCAP). It runs the prediction and returns a JSON response carrying the top label, confidence score, full per-class probability vector, per-flow breakdown, and processing time in milliseconds. Both interfaces share the same predictor pool and the same capture-to-CSV bridge, so they are functionally equivalent.

6.4.2 Frontend: NEURAL IDS React Dashboard

The React frontend connects to the REST endpoint and presents a polished cyberpunk-themed dashboard under the branding *NEURAL IDS*. Its main screen (Figure 6.2) presents three model-selection cards (MLP Neural Network, Random Forest, XGBoost) that set the `model_type` field of subsequent API calls, and an analysis history table that persists the timestamp, model label, filename, dominant predicted class, and confidence of every classification run, scoped to the signed-in user and retained across sessions (Section 6.4.4).

On submission, the frontend calls `POST /api/classify` with the selected model type, split, and mode parameters. The backend routes the request through the corresponding artefact triple and returns the prediction result. The frontend renders the result on a dedicated analysis screen (Figure 6.3) displaying a confidence gauge,

a ranked probability bar chart for all classes, per-request metadata (model, mode, split, flow count, processing time), and a flow-count breakdown by predicted label.

6.4.3 Two Symmetric Upload Paths

The CSV and PCAP upload paths are symmetric at the API level. Both paths reach the same inference component after the optional flow-extraction step. The CSV path validates column presence and forwards the frame directly; the PCAP path materialises the capture in a temporary directory, invokes the CICFlowMeter bridge to produce a feature CSV, and then takes the same downstream route. This shared post-extraction path means the JSON response schema and the frontend rendering logic are identical for both input types.

6.4.4 Authentication and Result Persistence

The deployed frontend communicates with two independent backends, and that separation is the central design decision of the serving tier (Figure 6.1). The first is the *stateless inference channel*: the React single-page application issues a cross-origin `POST /api/classify` to the FastAPI endpoint on Hugging Face Spaces (Section 6.4.1) and receives a JSON verdict. That backend holds no database and retains nothing about the user between requests. It is the artefact-driven predictor described throughout this chapter, merely reached over HTTPS. The second is the *stateful identity-and-history channel*: the same application talks directly to a managed Supabase project for authentication and for persisting the user's past results. Inference and identity are therefore fully decoupled: either backend can be replaced without touching the other, and an inference request never blocks on the persistence layer.

Authentication uses Supabase's email/password provider. On sign-in the client receives a JSON Web Token that is attached to subsequent storage requests, and the session is restored on reload from the persisted token. After each classification, the results screen writes a single row into the `analyses` table, tagged with the authenticated user's identifier, and the dashboard reads the same table back to render the history list. Each row holds only derived result metadata, never the uploaded capture or CSV: a surrogate UUID key, the owning `user_id`, a server-side timestamp, the classifier variant and classification granularity, the input modality (CSV or packet capture), the uploaded file name, the dominant predicted label with its calibrated confidence, and the full per-class probability vector as JSON. This keeps the persistence tier consistent with FR-D5. Per-user isolation is not enforced in client code: a row-level-security policy restricts every `select` and `insert` to rows whose `user_id` matches the caller's authenticated identifier, so one user can never read another's history even though all rows share a single table.

6.4.5 Public Deployment

The backend is packaged as a Docker image and deployed on Hugging Face Spaces, allowing the demo to be reached from any browser without the presenter's local machine being reachable. The React frontend is deployed separately on Vercel and communicates with the Spaces backend over HTTPS. For the local defence demonstration, the same backend process is run directly on the presenter's laptop and reached through the loopback address, with the Gradio tab serving as the fallback UI.

Figure 6.1 shows the resulting runtime topology. Vercel delivers the built single-page application to the browser; at runtime the application fans out to the two decoupled backends of Section 6.4.4. The diagram makes the key property explicit: the inference path (browser ↔ Hugging Face Spaces) and the identity-and-history path (browser ↔ Supabase) share no server-side state and can fail or be replaced independently.

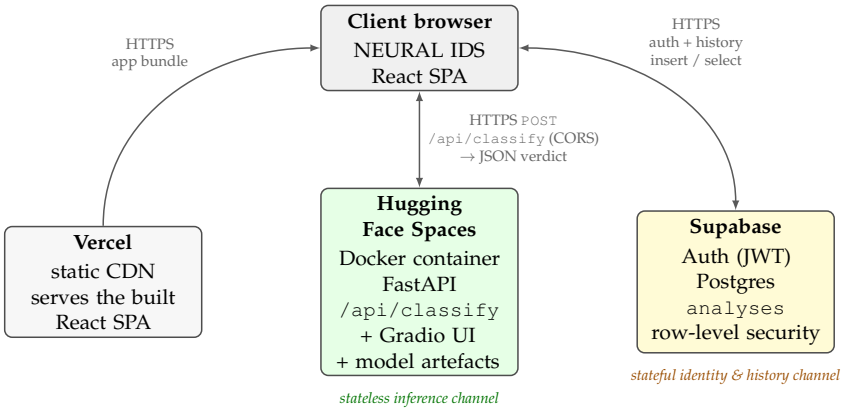

Figure 6.1: Runtime deployment topology of the serving subsystem. Vercel delivers the static React SPA (grey); at runtime the SPA drives two independent backends: a *stateless* inference service on Hugging Face Spaces (green) reached via `POST /api/classify`, and a *stateful* Supabase project (yellow) for authentication and per-user history. The two channels share no server-side state, so inference never blocks on persistence and either tier can be replaced in isolation.

6.5 Implementation Notes

Several implementation choices turn the design above into running code that respects the resource budgets. Memory discipline follows from the 16 GB budget (NFR-4): lazy frames are used wherever ingestion touches raw CSVs, with materialisation deferred until after column projection; the deduplicated, subsampled frame is the only materialised object on the data side, and its memory is reclaimed once it has been converted to numpy arrays, before the training tensors are allocated; and the cached intermediate is the only persistent caching boundary, with no in-memory cache

surviving a restart. Determinism follows from broadcasting a single seed to every random source, the numerical library, the deep-learning framework on CPU and GPU, the framework's deterministic backend flags, and the dataset samplers, so re-running against the same source files reproduces metrics within the few-thousandths-of-an-F1-point tolerance of NFR-3.

The remaining notes concern operability. The pipeline prints concise progress at every phase (per-folder ingestion counts, per-split row counts, missing-class warnings, per-epoch losses, and final per-class metrics) and renders training curves and confusion matrices inline, while the web application prints only its bind address and any per-request error. Configuration lives in a small block of named constants near the top of the notebook (the per-class cap, the active splits and granularities, the batch size, the maximum epochs, the early-stopping patience, the learning rate, and the seed), with a quick iteration mode (one split, one granularity) and a full thesis-run mode documented alongside so iteration cost is predictable. The runtime dependencies stay within the standard scientific-Python stack (a lazy columnar engine, a deep-learning framework, a tabular machine-learning library, and plotting libraries) plus a lightweight web-UI framework and a gradient-boosted-trees implementation for the baselines; there is no serving framework, since the demo loads the model directly into the application process, and no packaging beyond a flat directory launched by a single command, which satisfies NFR-7.

6.6 Testing Strategy

The application is tested at three levels: shape-and-determinism sanity checks inside the training pipeline; integration smoke tests over each (split, granularity) artefact triple; and the functional acceptance test, which demonstrates the core functionality end-to-end.

6.6.1 Pipeline-Level Sanity Checks

The training pipeline contains assertions and printed sanity checks at every transition. An assertion confirms that the fine-grained label mapping covers every attack folder exactly once, guarding against a missing or duplicated entry, and a second assertion during ingestion confirms that no folder is left unmapped, guarding against a new attack folder being silently dropped. For each split, a check confirms that every fine-grained label appears at least once in the train partition, flagging any missing class with a warning. Per-folder row counts (raw, deduplicated, sampled) are printed during ingestion, so a sudden mismatch between the raw and deduplicated counts is an immediate cue that the source data has changed.

6.6.2 Artefact Round-Trip Test

For every (split, granularity) pair, after training, the pipeline reloads the freshly written artefacts using the exact code path the demo uses and predicts on the test partition. The reloaded predictions are compared against the in-memory predictions of the still-loaded model. The two must agree exactly. This test caught two real bugs during development: a column-ordering drift between training and inference, and a silent dtype mismatch when arrays were converted to tensors without an explicit precision cast.

6.6.3 Functional Acceptance Test: Capture to Verdict

The core functionality is verified end-to-end with two test inputs whose ground-truth labels are known, so the test is a binary pass/fail rather than a subjective inspection. Each test runs the demo, uploads the input, and reads the predicted-label distribution. The benign baseline is a short capture of normal browsing traffic produced on the demo machine, and it passes if at least 95% of flows are predicted Benign at the binary granularity. The synthetic flood is a capture of a SYN-flood attack against a local target generated with a standard packet-crafting tool, and it passes if at least 95% of flows are predicted Attack at the binary granularity and the dominant predicted family is DDoS at the attack-family granularity.

These thresholds sit well below the test-set numbers (typically 0.97+ in the binary case), so the test absorbs the distribution shift between the training data and the demo machine while still failing loudly if the model has been deployed against the wrong scaler or encoder.

6.6.4 Negative-Path Testing

Three negative scenarios are exercised manually from the public component surface, so they reproduce from a brief interactive session. Uploading a CSV missing one or more expected feature columns should produce a domain-level error naming the missing columns, surfaced as the summary message with an empty table, which verifies FR-D4. Uploading a non-capture file (for example a text file renamed to a capture extension) should produce a flow-extraction error carrying the tool's stderr, again surfaced as the summary message, which also verifies FR-D4. Running the demo with no artefacts on disk should exit quickly with a message pointing the user back to the training pipeline, which verifies graceful behaviour when discovery yields nothing (FR-D2).

6.7 Deployment and Runtime Behaviour

The serving subsystem runs as a single Python process inside the project's virtual environment. The packet-capture path additionally needs a CICFlowMeter backend reachable from the process, either the Java reference implementation located through the `CICFLOWMETER_HOME` environment variable or the Python community port located through the executable search path, whereas the CSV path needs neither, since CICFlowMeter has already been applied upstream. On startup the application runs the discovery probe of Section 6.4.1, populates the variant dropdown with every artefact triple on disk, and defaults to the first one returned; the temporal-split binary variant is the one used for the acceptance evidence in Section 6.8, and the dropdown lets the same input be re-classified under any other variant. A request then follows one of two symmetric paths that converge on the same downstream code: the CSV path reads the file and validates it against the expected columns, while the packet-capture path materialises the `.pcap` or `.pcapng` file in a temporary directory and invokes the capture-to-CSV bridge to produce a feature CSV. In both cases the response is a summary line of the form "N flows classified, LabelA: n_A — LabelB: n_B — . . . " sorted by descending count, together with a per-flow verdict table giving the row index, the predicted label, and the maximum-softmax confidence to four decimal digits. Response time is dominated by flow extraction on the packet-capture path and by scaler-plus-forward latency on the CSV path, both of which fall inside the NFR-1 budget for the captures used as acceptance evidence.

6.8 Functional Acceptance Evidence

The functional acceptance test specified in Section 6.6.3 was executed against the deployed serving subsystem using both the local Gradio interface and the NEURAL IDS React dashboard. Two pre-extracted feature CSVs with known ground-truth labels were submitted: `demo_benign.csv` (normal browsing traffic captured on the demo machine and converted to CIC features) and `demo_syn_flood.csv` (a SYN-flood attack directed at a local target, converted via the same flow-extraction tool). Figure 6.2 shows the NEURAL IDS dashboard after all four acceptance runs have been completed, with the full analysis history visible in the main panel.

The history confirms both halves of the acceptance criterion. For `demo_syn_flood.csv`, the binary temporal-split MLP labelled every flow (100%) as Attack with a mean calibrated confidence of 99.6%, and a subsequent run of the same file under the 8-class variant confirmed DoS as the dominant family, both correct. For `demo_benign.csv`, the binary MLP returned Benign on 98% of flows (mean dominant-class confidence 71.9%), so the fraction labelled Benign exceeds the 95% threshold defined in Sec-

tion 6.6.3, discharging the no-false-alarm half of the criterion. (Confidence values report the mean softmax probability of the dominant class after temperature scaling, not the fraction of flows in that class.)

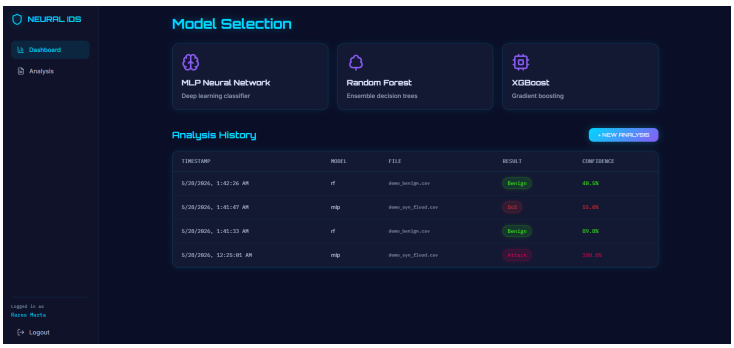

Figure 6.2: NEURAL IDS dashboard after the acceptance runs. The analysis history table records four classification requests: the SYN-flood capture classified as Attack (near-certain binary confidence) and as DoS (≈55%) under different granularities, and the benign capture classified as Benign. The three model-selection cards at the top set the `model_type` field of subsequent API calls. (Screenshot is illustrative; binary confidence is reported after temperature scaling in the text.)

Figure 6.3 shows the detailed result screen for the SYN-flood capture classified under the temporal-split 8-class Random Forest variant. The top predicted class is **DoS** at 55% mean confidence, with **DDoS** as the second-ranked class at 45%. This is the expected result: a SYN flood is structurally a single-source volumetric attack, which the model correctly places in the DoS family (single-source high-rate flood) rather than DDoS (distributed multi-source flood). The processing time of 8 ms for 50 flows is well within the NFR-1 budget of 100 ms at p95.

Figure 6.3: Functional acceptance: SYN-flood capture classified under the temporal-split 8-class Random Forest variant. The alert banner confirms a threat detection; the class-probability panel ranks DoS (55%) above DDoS (45%), correctly identifying the single-source flood family. Analysis metadata (50 flows, 8 ms processing time) are shown in the lower panel.

Chapter 7

Conclusions and Future Work

7.1 Summary of Contributions

This thesis designed, implemented, and evaluated a real-time intrusion detection system for IoT networks built on the CICIoT2023 dataset, delivering a reproducible offline training pipeline and an interactive web demonstration. The data pipeline ingests the raw CICIoT2023 captures, removes the large exact-duplicate fraction before sampling, subsamples each class to a 200,000-row cap, and writes a compressed columnar cache, reproducing byte-identical splits from the same seed and source files. A feature analysis characterises the 39 public features through variance and inter-feature correlation, pruning 14 near-zero-variance indicators and two perfectly collinear continuous pairs to leave a 25-feature input. On that input a two-hidden-layer MLP was trained at binary and 8-class attack-family granularities on the temporal split and compared against Random Forest and XGBoost baselines under identical splits and metrics, so the comparison is controlled. The MLP's softmax confidences were calibrated by temperature scaling at no cost to predicted labels. The NEURAL IDS web demonstration pairs a Gradio/FastAPI backend with a React dashboard that accepts pre-extracted CSVs or raw packet captures, classifies flows in under 10 ms, and shows confidence scores, per-class probabilities, and a timestamped history, deployed on Hugging Face Spaces for remote access. The quantitative outcomes of these experiments are summarised in the next section.

7.2 Key Findings

The experiments yield four main findings. The temporal split, in which the test captures are chronologically later than the training captures, gives a realistic measure of generalisation, and all three models generalise well (train-to-test accuracy drop below 8 percentage points), which indicates that CICIoT2023 sessions are reasonably

stationary within each attack folder; the binary temporal weighted F1 (≈0.90 for the MLP) clears the NFR-2 bar, while the 8-class figure (≈0.73 for the MLP, ≈0.77 for the tree ensembles) sits well below it, the expected price of fine-grained family discrimination when the features describe only transport-layer behaviour.

Mirai is trivially detectable, reaching near-perfect recall (0.99) thanks to its distinctive high-rate, fixed-payload traffic, whereas DDoS and DoS are frequently confused with each other (up to 36% cross-confusion) because both are volumetric floods separated mainly by the number of coordinated source hosts, a distinction the fixed-window feature representation does not capture reliably. The hardest classes are the application-layer attacks: Spoofing and BruteForce are weakest at 0.46 recall, Reconnaissance reaches 0.51, and the web family 0.52, with a large fraction of Spoofing flows classified as Benign. These families are distinguished mainly by payload content, which the header-and-metadata features do not capture; the ceiling here is set by the data, not the classifier. Finally, XGBoost and Random Forest outperform the MLP by roughly 4 to 5 percentage points on test accuracy and weighted F1, consistent with the tabular-learning literature on heterogeneous, non-spatial features at sample sizes in the low millions; the MLP is retained as the serving model because its inference is a single portable PyTorch call and the gap is small enough to keep the system within its targets.

7.3 Limitations

Several limitations bound the scope of these conclusions. The 25 retained features come from packet headers and metadata only, with no application-layer content (HTTP bodies, DNS query strings, TLS certificate fields), which caps detection accuracy for application-layer attacks and means the results do not generalise to settings where the attack signal lives in the payload. The captures come from a controlled testbed of 105 real IoT devices, so the attacker-victim topology, the absence of background noise, and the isolated execution of individual attack types do not reflect multi-stage campaigns, lateral movement, or the traffic diversity of a production network. Of the three split strategies designed (temporal, per-capture, random), only the temporal split is reported as the headline; the per-capture split, which removes within-session redundancy across partition boundaries, could give a complementary view of generalisation that the temporal split alone does not. The work uses the 39-feature public CSV release, and the seven features missing from it (source rate, destination rate, URG count, Magnitude, Radius, Covariance, Weight) may carry discriminative information, particularly for the weaker families, so the numbers here are not directly comparable to studies using the full 46-feature variant. Finally, temperature scaling reduces but does not eliminate calibration error (8-class ECE

0.089 after scaling): a single global temperature cannot correct per-class miscalibration, and calibration was measured in-distribution, so behaviour on genuinely out-of-distribution traffic, where softmax outputs are known to be overconfident [6], remains uncharacterised.

7.4 Future Work

The findings and limitations above suggest several directions for future work. The most immediate is to complete the evaluation on the per-capture split, which would give a cleaner measure of within-session redundancy and clarify how much of the temporal split's generalisation gap comes from temporal ordering as opposed to session-level diversity. A second is to rerun the pipeline on the full 46-feature variant once it is available from a verified source, allowing a direct comparison with published baselines and possibly improving detection of the weaker families. A third is to move beyond the global temperature scaling already applied (Section 4.8) towards per-class scaling and per-class decision thresholds, so the detection-rate and false-alarm trade-off can be tuned per attack family (for instance a lower threshold for the low-recall Spoofing class) rather than fixed at the argmax. A fourth is to extend the feature set with TLS-visible metadata such as certificate fields, SNI, and session-length distributions, which would reach partially encrypted traffic and may help the web-based attacks that the transport-layer representation currently limits. Beyond these, deploying the serving component on a real IoT gateway with a periodic retraining schedule driven by misclassified flows would test the pipeline's generalisation outside the laboratory and address the static, offline nature of the present model.

Bibliography

[1] Hassan M. J. Almohri et al. Explainable artificial intelligence for intrusion detection in iot networks: A deep learning based approach. *Expert Systems with Applications*, 238:121855, 2024.

[2] Moayad Aloqaily, Safa Otoum, Ismaeel Al Ridhawi, and Yaser Jararweh. Ai-ids: Application of deep learning to realtime web intrusion detection. In *2020 International Wireless Communications and Mobile Computing (IWCMC)*, pages 2058–2063. IEEE, 2020.

[3] Giuseppina Andresini, Feargus Pendlebury, Fabio Pierazzi, Corrado Loglisci, Annalisa Appice, and Lorenzo Cavallaro. INSOMNIA: Towards concept-drift robustness in network intrusion detection. In *Proceedings of the 14th ACM Workshop on Artificial Intelligence and Security (AISec '21)*, pages 111–122. ACM, 2021.

[4] Giovanni Apruzzese, Pavel Laskov, and Johannes Schneider. SoK: Pragmatic assessment of machine learning for network intrusion detection. In *Proceedings of the 8th IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 592–614, Delft, Netherlands, 2023. IEEE.

[5] Mohamed Amine Ferrag, Leandros Maglaras, Sotiris Moschoyiannis, and Helge Janicke. Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study. *Journal of Information Security and Applications*, 50:102419, 2020.

[6] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, volume 70, pages 1321–1330, Sydney, Australia, 2017. PMLR.

[7] Arlind Kadra, Marius Lindauer, Frank Hutter, and Josif Grabocka. Well-tuned simple nets excel on tabular datasets. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, pages 23928–23941, 2021.

[8] Jiyeon Kim, Jiwon Kim, Hyunjung Kim, Minsun Shim, and Eunjung Choi. CNN-based network intrusion detection against denial-of-service attacks. *Electronics*, 9(6):916, 2020.

[9] Jan Lansky, Saqib Ali, Mokhtar Mohammadi, Mohammed Kamal Majeed, Sarkhel H. Taher Karim, Shima Rashidi, Mehdi Hosseinzadeh, and Amir Masoud Rahmani. Deep learning-based intrusion detection systems: A systematic review. *IEEE Access*, 9:101574–101599, 2021.

[10] Yisroel Mirsky, Tomer Doitshman, Yuval Elovici, and Asaf Shabtai. Kitsune: An ensemble of autoencoders for online network intrusion detection. In *Proceedings of the 25th Annual Network and Distributed System Security Symposium (NDSS)*, San Diego, CA, USA, 2018. Internet Society.

[11] Muhammad Nadeem et al. Ciciot2023: A real-time dataset and benchmark for large-scale attacks in iot environment. *Sensors*, 23(13):5941, 2023.

[12] Martin Roesch. Snort – lightweight intrusion detection for networks. In *Proceedings of the 13th USENIX Conference on System Administration (LISA '99)*, pages 229–238, Seattle, Washington, USA, 1999.

[13] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pages 108–116, 2018.

[14] R. Vinayakumar, Mamoun Alazab, K. P. Soman, Prabaharan Poornachandran, Ameer Al-Nemrat, and Sitalakshmi Venkatraman. Deep learning approach for intelligent intrusion detection system. *IEEE Access*, 7:41525–41550, 2019.

[15] R. Vinayakumar, K. P. Soman, and Prabaharan Poornachandran. Applying convolutional neural network for network intrusion detection. In *2017 International Conference on Advances in Computing, Communications and Informatics (ICACCI)*, pages 1222–1228. IEEE, 2017. Frequently cited under year 2018 due to indexing date.

[16] Zihan Wu, Hong Zhang, Penghai Wang, and Zhibo Sun. RTIDS: A robust transformer-based approach for intrusion detection system. *IEEE Access*, 10:64375–64387, 2022.